🚀 Proyecto MIA: Segmentación Inteligente (Versión Infalible)

```
# @title 🚀 Proyecto MIA: Segmentación Inteligente (Versión Infalible)
# Esta versión incluye un respaldo automático si el botón de carga falla.

import sys
import subprocess
import pkg_resources

# 1. INSTALACIÓN DE LIBRERÍAS
def install_packages():
    required = {'plotly', 'pandas', 'numpy', 'scikit-learn', 'ipywidgets'}
    installed = {pkg.key for pkg in pkg_resources.working_set}
    missing = required - installed
    if missing:
        print(f"⚙️ Instalando librerías necesarias...")
        try:
            subprocess.check_call([sys.executable, '-m', 'pip', 'install', *missing, '-q'])
            print("✅ Instalación completada.")
        except: pass

install_packages()

# Habilitar widgets
try:
    from google.colab import output
    output.enable_custom_widget_manager()
except: pass

import pandas as pd
import numpy as np
import datetime as dt
import plotly.express as px
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import io
from google.colab import files  # IMPORTANTE: Para el respaldo nativo

# 2. ESTILOS VISUALES (TEMA TESIS)
style = """
<style>
    :root { --primary: #4f46e5; --bg: #0f172a; --card: #1e293b; --text: #f8fafc; }
    .custom-card { background-color: var(--card); border-radius: 12px; padding: 20px; margin-bottom: 20px; border: 1px solid #334155; }
    .metric-val { font-size: 2rem; font-weight: 700; color: #38bdf8; }
    .metric-lbl { font-size: 0.8rem; color: #94a3b8; text-transform: uppercase; letter-spacing: 1px; }
    h1 { color: #818cf8; margin-bottom: 5px; }
    .jupyter-widgets { color: #333; }
</style>
"""
display(HTML(style))

# 3. MOTOR ANALÍTICO
class MIAEngine:
    def __init__(self):
        self.rfm = None

    def process(self, file_content):
        try:
            # Leer CSV
            df = pd.read_csv(io.BytesIO(file_content))

            # Normalizar columnas (evita errores por mayúsculas/espacios)
            df.columns = [c.lower().strip() for c in df.columns]
            rename_map = {
                'transaction id': 'invoiceno', 'date': 'invoicedate', 'customer id': 'customerid',
                'quantity': 'quantity', 'price per unit': 'unitprice', 'total amount': 'total_amount'
            }
            df.rename(columns=rename_map, inplace=True)

            # Validar columnas mínimas
            req_cols = {'invoiceno', 'invoicedate', 'customerid', 'quantity', 'unitprice', 'total_amount'}
            if not req_cols.issubset(df.columns):
                return None, f"Faltan columnas requeridas. Tu archivo tiene: {list(df.columns)}"

            # Limpieza y Conversión
            df['invoicedate'] = pd.to_datetime(df['invoicedate'], errors='coerce')
            df['total_amount'] = pd.to_numeric(df['total_amount'], errors='coerce')
            df = df.dropna(subset=['invoicedate', 'customerid'])
            df = df[df['quantity'] > 0] # Eliminar devoluciones

            # Cálculo RFM (Recencia, Frecuencia, Monto)
            now = df['invoicedate'].max() + dt.timedelta(days=1)
            rfm = df.groupby('customerid').agg({
```

```
                    'invoicedate': lambda x: (now - x.max()).days,
                    'invoiceno': 'nunique',
                    'total_amount': 'sum'
                }).rename(columns={'invoicedate': 'Recency', 'invoiceno': 'Frequency', 'total_amount': 'Monetary'})

                # K-Means Clustering
                scaler = StandardScaler()
                rfm_scaled = scaler.fit_transform(np.log1p(rfm)) # Log para normalizar distribución
                kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
                rfm['Cluster'] = kmeans.fit_predict(rfm_scaled)

                # Ranking y Etiquetado de Segmentos
                score = rfm.groupby('Cluster').mean()
                # Score simple: (Frecuencia + Monto) / Recencia
                score['val'] = (score['Frequency'] + score['Monetary']) / (score['Recency'] + 1)
                rank = score.sort_values('val', ascending=False).index.tolist()

                labels = {rank[0]: 'Campeones (VIP)', rank[1]: 'Leales', rank[2]: 'Necesitan Atención', rank[3]: 'En Riesgo'}
                rfm['Segmento'] = rfm['Cluster'].map(labels)

                self.rfm = rfm
                return rfm, "OK"
            except Exception as e:
                return None, str(e)

    engine = MIAEngine()

    # 4. INTERFAZ GRÁFICA
    btn_upload = widgets.FileUpload(accept='.csv', multiple=False, description='1. Cargar CSV')
    btn_process = widgets.Button(description='2. Procesar e Informe', button_style='success', icon='cogs')
    dd_segment = widgets.Dropdown(description='Filtro:', disabled=True)
    out = widgets.Output()

    def show_dashboard(rfm):
        with out:
            clear_output()
            seg = dd_segment.value
            if not seg: return

            subset = rfm[rfm['Segmento'] == seg]

            # TARJETAS KPI
            display(HTML(f"""
            <div style="display:flex; gap:20px; margin-bottom:20px;">
                <div class="custom-card" style="flex:1"><div class="metric-lbl">Días desde última compra</div><div class="metric-val">{subset['Recency'].mean():.0f}</div></div>
                <div class="custom-card" style="flex:1"><div class="metric-lbl">Frecuencia de Compra</div><div class="metric-val">{subset['Frequency'].mean():.1f}</div></div>
                <div class="custom-card" style="flex:1"><div class="metric-lbl">Valor Monetario Promedio</div><div class="metric-val">${subset['Monetary'].mean():.0f}</div></div>
            </div>"""))

            # GRÁFICOS
            colors = {'Campeones (VIP)': '#10b981', 'Leales': '#3b82f6', 'Necesitan Atención': '#f59e0b', 'En Riesgo': '#ef4444'}

            # Barras (Distribución)
            counts = rfm['Segmento'].value_counts().reset_index()
            counts.columns = ['Segmento', 'Clientes']
            counts['alpha'] = counts['Segmento'].apply(lambda x: 1 if x == seg else 0.3)
            fig_bar = px.bar(counts, x='Segmento', y='Clientes', color='Segmento',
                             color_discrete_map=colors, opacity=counts['alpha'], template='plotly_dark',
                             title="Distribución de Clientes")
            fig_bar.update_layout(height=350, showlegend=False, margin=dict(l=20,r=20,t=40,b=20))

            # Scatter (Mapa)
            fig_scat = px.scatter(rfm, x='Recency', y='Monetary', color='Segmento', log_y=True,
                                  color_discrete_map=colors, template='plotly_dark', title=f'Mapa de Segmentos: {seg}')
            fig_scat.update_traces(marker=dict(size=6, opacity=0.2)) # Fondo tenue
            fig_scat.update_traces(selector=dict(name=seg), marker=dict(size=10, opacity=1, line=dict(width=1, color='white'))) # Resaltado
            fig_scat.update_layout(height=350, margin=dict(l=20,r=20,t=40,b=20))

            o1, o2 = widgets.Output(), widgets.Output()
            with o1: fig_bar.show()
            with o2: fig_scat.show()
            display(widgets.HBox([o1, o2]))

            # ESTRATEGIA (TEXTO)
            strategies = {
                'Campeones (VIP)': ('Fidelización VIP', 'Acceso Anticipado a Nuevos Productos', 'Email Exclusivo'),
                'Leales': ('Aumentar Ticket', 'Descuentos por Volumen de Compra', 'Notificación App'),
                'Necesitan Atención': ('Reactivación', 'Cupón de Descuento Limitado', 'Email Automatizado'),
                'En Riesgo': ('Recuperación', 'Promoción Agresiva de Retorno', 'SMS / WhatsApp')
            }
            strat = strategies.get(seg, ('-', '-', '-'))

            display(HTML(f"""
            <div class="custom-card">
                <h3 style="color:#818cf8; margin-top:0;">🎯 Estrategia Recomendada: {seg}</h3>
                <table style="width:100%; color:#cbd5e1; text-align:left; border-collapse: collapse;">
                    <tr style="border-bottom:1px solid #475569"><th style="padding:10px">Objetivo Táctico</th><th style="padding:10px">Acción Sugerida</th><th style="padding:10px">Canal Óptimo</th></
```

```
                <tr><td style="padding:10px">{strat[0]}</td><td style="padding:10px"><b>{strat[1]}</b></td><td style="padding:10px">{strat[2]}</td></tr>
            </table>
        </div>
        """))

def on_click_process(b):
    with out:
        clear_output()
        print("🔄 Iniciando análisis...")

        content = None

        # INTENTO 1: Widget de carga
        if btn_upload.value:
            try:
                vals = btn_upload.value
                # Lógica robusta para extraer contenido según versión de ipywidgets
                if isinstance(vals, tuple): content = vals[0].get('content')
                elif isinstance(vals, dict): content = list(vals.values())[0].get('content')
                elif isinstance(vals, list): content = vals[0].get('content')

                if content: print("✅ Archivo detectado desde el botón.")
            except: pass

        # INTENTO 2: Respaldo nativo de Colab (Si falla el widget)
        if content is None:
            print("⚠️ No se detectó archivo en el botón (o hubo error de sincronización).")
            print("📂 Abriendo selector de archivos nativo de Colab como respaldo...")
            try:
                uploaded = files.upload()
                if uploaded:
                    filename = list(uploaded.keys())[0]
                    content = uploaded[filename]
                    print(f"✅ Archivo '{filename}' cargado correctamente.")
                else:
                    print("❌ Cancelado por el usuario.")
                    return
            except Exception as e:
                print(f"❌ Error en carga nativa: {e}")
                return

        # Procesamiento
        if isinstance(content, memoryview): content = content.tobytes()

        rfm, msg = engine.process(content)
        if rfm is not None:
            opts = sorted(rfm['Segmento'].unique())
            dd_segment.options = opts
            dd_segment.value = opts[0]
            dd_segment.disabled = False
            print("✅ Segmentación completada con éxito.")
            show_dashboard(rfm)
        else:
            print(f"❌ Error procesando el archivo: {msg}")

def on_change_segment(change):
    if change['type'] == 'change' and change['name'] == 'value':
        show_dashboard(engine.rfm)

btn_process.on_click(on_click_process)
dd_segment.observe(on_change_segment)

# Layout Principal
header = widgets.HTML("<h1>🚀 Sistema de Segmentación Inteligente</h1><p>Herramienta de análisis para defensa de tesis.</p>")
controls = widgets.HBox([btn_upload, btn_process, dd_segment])
display(widgets.VBox([header, controls, out]))
```

```
/tmp/ipython-input-2797060419.py:6: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  import pkg_resources
```

🚀 **Sistema de Segmentación Inteligente**

Herramienta de análisis para defensa de tesis.

1. Cargar CSV (1) | 2. Procesar e Inf… | Filtro: Campeones (VIP)

🚀 Proyecto MIA: Segmentación Inteligente (Versión Defensa Final V2)

```
# @title 🚀 Proyecto MIA: Segmentación Inteligente (Versión Defensa Final V2)
# Corrección: Asegura que el dashboard se muestre tras la carga manual.

import sys
import subprocess
import pkg_resources

# 1. INSTALACIÓN DE LIBRERÍAS
```

```
def install_packages():
    required = {'plotly', 'pandas', 'numpy', 'scikit-learn', 'ipywidgets'}
    installed = {pkg.key for pkg in pkg_resources.working_set}
    missing = required - installed
    if missing:
        print(f"⚙️ Instalando librerías necesarias...")
        try:
            subprocess.check_call([sys.executable, '-m', 'pip', 'install', *missing, '-q'])
            print("✅ Instalación completada.")
        except: pass

install_packages()

# Habilitar widgets
try:
    from google.colab import output
    output.enable_custom_widget_manager()
except: pass

import pandas as pd
import numpy as np
import datetime as dt
import plotly.express as px
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import io
from google.colab import files

# 2. ESTILOS VISUALES
style = """
<style>
    :root { --primary: #4f46e5; --bg: #0f172a; --card: #1e293b; --text: #f8fafc; }
    .custom-card { background-color: var(--card); border-radius: 12px; padding: 20px; margin-bottom: 20px; border: 1px solid #334155; }
    .metric-val { font-size: 2rem; font-weight: 700; color: #38bdf8; }
    .metric-lbl { font-size: 0.8rem; color: #94a3b8; text-transform: uppercase; letter-spacing: 1px; }
    h1 { color: #818cf8; margin-bottom: 5px; }
    .jupyter-widgets { color: #333; }
</style>
"""
display(HTML(style))

# 3. MOTOR ANALÍTICO
class MIAEngine:
    def __init__(self):
        self.rfm = None

    def process(self, file_content):
        try:
            df = pd.read_csv(io.BytesIO(file_content))
            df.columns = [c.lower().strip() for c in df.columns]
            rename_map = {
                'transaction id': 'invoiceno', 'date': 'invoicedate', 'customer id': 'customerid',
                'quantity': 'quantity', 'price per unit': 'unitprice', 'total amount': 'total_amount'
            }
            df.rename(columns=rename_map, inplace=True)

            req_cols = {'invoiceno', 'invoicedate', 'customerid', 'quantity', 'unitprice', 'total_amount'}
            if not req_cols.issubset(df.columns):
                return None, f"Faltan columnas requeridas. Tu archivo tiene: {list(df.columns)}"

            df['invoicedate'] = pd.to_datetime(df['invoicedate'], errors='coerce')
            df['total_amount'] = pd.to_numeric(df['total_amount'], errors='coerce')
            df = df.dropna(subset=['invoicedate', 'customerid'])
            df = df[df['quantity'] > 0]

            now = df['invoicedate'].max() + dt.timedelta(days=1)
            rfm = df.groupby('customerid').agg({
                'invoicedate': lambda x: (now - x.max()).days,
                'invoiceno': 'nunique',
                'total_amount': 'sum'
            }).rename(columns={'invoicedate': 'Recency', 'invoiceno': 'Frequency', 'total_amount': 'Monetary'})

            scaler = StandardScaler()
            rfm_scaled = scaler.fit_transform(np.log1p(rfm))
            kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
            rfm['Cluster'] = kmeans.fit_predict(rfm_scaled)

            score = rfm.groupby('Cluster').mean()
            score['val'] = (score['Frequency'] + score['Monetary']) / (score['Recency'] + 1)
            rank = score.sort_values('val', ascending=False).index.tolist()

            labels = {rank[0]: 'Campeones (VIP)', rank[1]: 'Leales', rank[2]: 'Necesitan Atención', rank[3]: 'En Riesgo'}
            rfm['Segmento'] = rfm['Cluster'].map(labels)

            self.rfm = rfm
```

```
                return rfm, "OK"
            except Exception as e:
                return None, str(e)

    engine = MIAEngine()

    # 4. INTERFAZ GRÁFICA
    btn_upload = widgets.FileUpload(accept='.csv', multiple=False, description='1. Cargar CSV')
    btn_process = widgets.Button(description='2. Procesar e Informe', button_style='success', icon='cogs')
    dd_segment = widgets.Dropdown(description='Filtro:', disabled=True)
    out = widgets.Output()

    def show_dashboard(rfm):
        with out:
            clear_output()

            # Si no hay segmento seleccionado, tomar el primero
            if not dd_segment.value and not dd_segment.options:
                 opts = sorted(rfm['Segmento'].unique())
                 dd_segment.options = opts
                 dd_segment.value = opts[0]
                 dd_segment.disabled = False

            seg = dd_segment.value
            if not seg: return

            subset = rfm[rfm['Segmento'] == seg]

            # TARJETAS KPI
            display(HTML(f"""
            <div style="display:flex; gap:20px; margin-bottom:20px;">
                <div class="custom-card" style="flex:1"><div class="metric-lbl">Recencia Promedio</div><div class="metric-val">{subset['Recency'].mean():.0f} días</div></div>
                <div class="custom-card" style="flex:1"><div class="metric-lbl">Frecuencia Promedio</div><div class="metric-val">{subset['Frequency'].mean():.1f} compras</div></div>
                <div class="custom-card" style="flex:1"><div class="metric-lbl">Ticket Promedio</div><div class="metric-val">${subset['Monetary'].mean():.0f}</div></div>
            </div>"""))

            # GRÁFICOS
            colors = {'Campeones (VIP)': '#10b981', 'Leales': '#3b82f6', 'Necesitan Atención': '#f59e0b', 'En Riesgo': '#ef4444'}

            # Barras (Distribución)
            counts = rfm['Segmento'].value_counts().reset_index()
            counts.columns = ['Segmento', 'Clientes']
            counts['alpha'] = counts['Segmento'].apply(lambda x: 1 if x == seg else 0.3)
            fig_bar = px.bar(counts, x='Segmento', y='Clientes', color='Segmento',
                             color_discrete_map=colors, opacity=counts['alpha'], template='plotly_dark',
                             title="Distribución de Clientes por Segmento")
            fig_bar.update_layout(height=350, showlegend=False, margin=dict(l=20,r=20,t=40,b=20))

            # Scatter (Mapa)
            fig_scat = px.scatter(rfm, x='Recency', y='Monetary', color='Segmento', log_y=True,
                                  color_discrete_map=colors, template='plotly_dark', title=f'Mapa de Segmentos: {seg}')
            fig_scat.update_traces(marker=dict(size=6, opacity=0.2))
            fig_scat.update_traces(selector=dict(name=seg), marker=dict(size=10, opacity=1, line=dict(width=1, color='white')))
            fig_scat.update_layout(height=350, margin=dict(l=20,r=20,t=40,b=20))

            o1, o2 = widgets.Output(), widgets.Output()
            with o1: fig_bar.show()
            with o2: fig_scat.show()
            display(widgets.HBox([o1, o2]))

            # ESTRATEGIA (TEXTO)
            strategies = {
                'Campeones (VIP)': ('Fidelización VIP', 'Acceso Anticipado a Nuevos Productos', 'Email Exclusivo'),
                'Leales': ('Aumentar Ticket', 'Descuentos por Volumen de Compra', 'Notificación App'),
                'Necesitan Atención': ('Reactivación', 'Cupón de Descuento Limitado', 'Email Automatizado'),
                'En Riesgo': ('Recuperación', 'Promoción Agresiva de Retorno', 'SMS / WhatsApp')
            }
            strat = strategies.get(seg, ('-', '-', '-'))

            display(HTML(f"""
            <div class="custom-card">
                <h3 style="color:#818cf8; margin-top:0;">🎯 Estrategia Recomendada: {seg}</h3>
                <table style="width:100%; color:#cbd5e1; text-align:left; border-collapse: collapse;">
                    <tr style="border-bottom:1px solid #475569"><th style="padding:10px">Objetivo Táctico</th><th style="padding:10px">Acción Sugerida</th><th style="padding:10px">Canal Óptimo</th></tr>
                    <tr><td style="padding:10px">{strat[0]}</td><td style="padding:10px"><b>{strat[1]}</b></td><td style="padding:10px">{strat[2]}</td></tr>
                </table>
            </div>
            """))

    def on_click_process(b):
        with out:
            clear_output()
            print("🔍 Iniciando análisis...")

            content = None

            # INTENTO 1: Widget de carga
```

```
        if btn_upload.value:
            try:
                vals = btn_upload.value
                if isinstance(vals, tuple): content = vals[0].get('content')
                elif isinstance(vals, dict): content = list(vals.values())[0].get('content')
                elif isinstance(vals, list): content = vals[0].get('content')
                if content: print("✅ Archivo detectado desde el botón.")
            except: pass

        # INTENTO 2: Respaldo nativo de Colab (Si falla el widget)
        if content is None:
            print("⚠️ No se detectó archivo en el botón.")
            print("📂 Abriendo selector de archivos nativo de Colab...")
            try:
                uploaded = files.upload()
                if uploaded:
                    filename = list(uploaded.keys())[0]
                    content = uploaded[filename]
                    print(f"✅ Archivo '{filename}' cargado correctamente.")
                else:
                    print("❌ Cancelado por el usuario.")
                    return
            except Exception as e:
                print(f"❌ Error en carga nativa: {e}")
                return

        # Procesamiento
        if isinstance(content, memoryview): content = content.tobytes()

        rfm, msg = engine.process(content)
        if rfm is not None:
            # Configurar Dropdown ANTES de mostrar dashboard
            opts = sorted(rfm['Segmento'].unique())
            dd_segment.options = opts
            dd_segment.value = opts[0]
            dd_segment.disabled = False

            print("✅ Segmentación completada. Generando reporte visual...")
            show_dashboard(rfm) # <--- AQUÍ ESTÁ LA CLAVE: LLAMADA EXPLÍCITA
        else:
            print(f"❌ Error procesando el archivo: {msg}")

    def on_change_segment(change):
        if change['type'] == 'change' and change['name'] == 'value':
            if engine.rfm is not None:
                show_dashboard(engine.rfm)

btn_process.on_click(on_click_process)
dd_segment.observe(on_change_segment)

# Layout Principal
header = widgets.HTML("<h1>🚀 Sistema de Segmentación Inteligente</h1><p>Herramienta de análisis para defensa de tesis.</p>")
controls = widgets.HBox([btn_upload, btn_process, dd_segment])
display(widgets.VBox([header, controls, out]))
```

🚀 Sistema de Segmentación Inteligente

Herramienta de análisis para defensa de tesis.

1. Cargar CSV (1) | 2. Procesar e Inf… | Filtro: En Riesgo

🦄 Sistema de Segmentación Inteligente (Dash en Colab)

```
# @title 🦄 Sistema de Segmentación Inteligente (Dash en Colab)
# Ejecuta esta celda para iniciar la aplicación completa.

# 1. INSTALACIÓN SILENCIOSA
import sys
import subprocess
def install(package):
    subprocess.check_call([sys.executable, "-m", "pip", "install", package, "-q"])

print("⚙️ Instalando librerías necesarias... (Esto puede tardar un minuto)")
install("dash")
install("pandas")
install("numpy")
install("plotly")
install("scikit-learn")
# install("jupyter-dash") # Ya no es necesario

# 2. IMPORTACIONES
import base64
import datetime
import io
```

```
import pandas as pd
import numpy as np
from dash import Dash, dcc, html, Input, Output, State, dash_table, no_update
# from jupyter_dash import JupyterDash # Versión compatible con Colab - DEPRECATED
import plotly.express as px
import plotly.graph_objects as go
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# 3. LÓGICA DEL MOTOR (BACKEND)
class AnalysisEngine:
    def parse_contents(self, contents, filename):
        content_type, content_string = contents.split(',')
        decoded = base64.b64decode(content_string)
        try:
            if 'csv' in filename:
                df = pd.read_csv(io.StringIO(decoded.decode('utf-8')))
            elif 'xls' in filename:
                df = pd.read_excel(io.BytesIO(decoded))

            # Normalización
            df.columns = [c.lower().strip() for c in df.columns]
            rename_map = {
                'transaction id': 'invoiceno', 'date': 'invoicedate', 'customer id': 'customerid',
                'quantity': 'quantity', 'price per unit': 'unitprice', 'total amount': 'total_amount'
            }
            df.rename(columns=rename_map, inplace=True)
            return df
        except Exception as e:
            print(e)
            return None

    def process_rfm(self, df):
        # Limpieza
        df['invoicedate'] = pd.to_datetime(df['invoicedate'], errors='coerce')
        df['total_amount'] = pd.to_numeric(df['total_amount'], errors='coerce')
        df = df.dropna(subset=['invoicedate', 'customerid'])
        df = df[df['quantity'] > 0]

        # RFM
        now = df['invoicedate'].max() + datetime.timedelta(days=1)
        rfm = df.groupby('customerid').agg({
            'invoicedate': lambda x: (now - x.max()).days,
            'invoiceno': 'nunique',
            'total_amount': 'sum'
        }).rename(columns={'invoicedate': 'Recency', 'invoiceno': 'Frequency', 'total_amount': 'Monetary'})

        return rfm

    def run_kmeans(self, rfm, k=4):
        scaler = StandardScaler()
        rfm_log = np.log1p(rfm)
        rfm_scaled = scaler.fit_transform(rfm_log)

        kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
        rfm['Cluster'] = kmeans.fit_predict(rfm_scaled)

        # Ranking
        score = rfm.groupby('Cluster').mean()
        score['val'] = (score['Frequency'] + score['Monetary']) / (rfm['Recency'].mean() + 1)
        rank = score.sort_values('val', ascending=False).index.tolist()

        labels = {rank[0]: 'Campeones (VIP)', rank[1]: 'Leales', rank[2]: 'Necesitan Atención', rank[3]: 'En Riesgo'}
        # Manejo si k != 4
        if k != 4:
             labels = {c: f'Segmento {c}' for c in range(k)}

        rfm['Segmento'] = rfm['Cluster'].map(labels)
        return rfm

engine = AnalysisEngine()

# 4. CONFIGURACIÓN DE LA APP DASH
# Usamos Dash directamente para que corra dentro de Colab
app = Dash(__name__, external_stylesheets=['https://codepen.io/chriddyp/pen/bWLwgP.css'])

# Colores y Estilos
COLORS = {'bg': '#0f172a', 'card': '#1e293b', 'text': '#f8fafc', 'accent': '#6366f1'}
STYLE_CARD = {'backgroundColor': COLORS['card'], 'padding': '20px', 'borderRadius': '10px', 'boxShadow': '0 4px 6px rgba(0,0,0,0.1)', 'color': COLORS['text'], 'marginBottom': '20px'}

# Layout Principal
app.layout = html.Div(style={'backgroundColor': COLORS['bg'], 'minHeight': '100vh', 'padding': '20px', 'fontFamily': 'sans-serif'}, children=[

    # Encabezado
    html.Div([
        html.H1("🚀 Proyecto MIA: Segmentación Inteligente", style={'color': COLORS['accent'], 'textAlign': 'center'}),
        html.P("Plataforma de análisis de clientes basada en RFM y Machine Learning (K-Means)", style={'color': '#94a3b8', 'textAlign': 'center'}),
```

```
        ], style=STYLE_CARD),

        # Área de Carga
        html.Div([
            dcc.Upload(
                id='upload-data',
                children=html.Div(['Arrastra o ', html.A('Selecciona tu CSV')]),
                style=
{
                    'width': '100%', 'height': '60px', 'lineHeight': '60px',
                    'borderWidth': '1px', 'borderStyle': 'dashed', 'borderRadius': '5px',
                    'textAlign': 'center', 'margin': '10px', 'color': COLORS['text'], 'borderColor': COLORS['accent']
                },
                multiple=False
            ),
            html.Div(id='output-data-upload', style={'color': 'green', 'textAlign': 'center'}),
            # Almacenamiento oculto
            dcc.Store(id='store-rfm'),
        ], style=STYLE_CARD),

        # Área de Resultados (Oculta hasta cargar)
        html.Div(id='results-container', style={'display': 'none'}, children=[

            # Filtros y KPIs
            html.Div([
                html.Div([
                    html.Label("🔍 Filtrar por Segmento:", style={'color': COLORS['text'], 'fontWeight': 'bold'}),
                    dcc.Dropdown(id='segment-dropdown', style={'color': '#333'}),
                ], style={'width': '30%', 'display': 'inline-block', 'verticalAlign': 'top'}),

                html.Div([
                    html.Div(id='kpi-recency', className='three columns', style={'textAlign': 'center', 'color': '#38bdf8', 'fontSize': '20px', 'fontWeight': 'bold'}),
                    html.Div(id='kpi-freq', className='three columns', style={'textAlign': 'center', 'color': '#38bdf8', 'fontSize': '20px', 'fontWeight': 'bold'}),
                    html.Div(id='kpi-money', className='three columns', style={'textAlign': 'center', 'color': '#38bdf8', 'fontSize': '20px', 'fontWeight': 'bold'}),
                ], style={'width': '65%', 'display': 'inline-block', 'float': 'right'})
            ], style=STYLE_CARD),

            # Gráficos
            html.Div([
                html.Div([dcc.Graph(id='graph-scatter')], className='six columns'),
                html.Div([dcc.Graph(id='graph-bar')], className='six columns'),
            ], className='row', style={'marginBottom': '20px'}),

            # Estrategia
            html.Div([
                html.H3("🎯 Estrategia Recomendada", style={'color': COLORS['accent']}),
                html.Div(id='strategy-content')
            ], style=STYLE_CARD),

            # Tabla de Datos
            html.Div([
                html.H4("📋 Top Clientes del Segmento", style={'color': COLORS['text']}),
                dash_table.DataTable(
                    id='table-clients',
                    style_header={'backgroundColor': '#334155', 'color': 'white'},
                    style_cell={'backgroundColor': '#1e293b', 'color': '#e2e8f0', 'textAlign': 'left'},
                    page_size=10
                )
            ], style=STYLE_CARD)
        ])
    ])

    # 5. CALLBACKS (INTERACTIVIDAD)
    @app.callback(
        [Output('store-rfm', 'data'),
         Output('output-data-upload', 'children'),
         Output('results-container', 'style')],
        [Input('upload-data', 'contents')],
        [State('upload-data', 'filename')]
    )
    def update_output(contents, filename):
        if contents is None:
            return no_update, "", {'display': 'none'}

        df = engine.parse_contents(contents, filename)
        if df is not None:
            rfm = engine.process_rfm(df)
            rfm_segmented = engine.run_kmeans(rfm)
            # Convertir a dict para guardar en dcc.Store
            return rfm_segmented.reset_index().to_dict('records'), f"✅ Archivo '{filename}' procesado con éxito.", {'display': 'block'}
        return None, "❌ Error al procesar archivo.", {'display': 'none'}

    @app.callback(
        [Output('segment-dropdown', 'options'),
         Output('segment-dropdown', 'value')],
        [Input('store-rfm', 'data')]
    )
```

```
def update_dropdown(data):
    if data is None: return [], None
    df = pd.DataFrame(data)
    options = [{'label': s, 'value': s} for s in sorted(df['Segmento'].unique())]
    return options, options[0]['value']

@app.callback(
    [Output('kpi-recency', 'children'),
     Output('kpi-freq', 'children'),
     Output('kpi-money', 'children'),
     Output('graph-scatter', 'figure'),
     Output('graph-bar', 'figure'),
     Output('strategy-content', 'children'),
     Output('table-clients', 'data'),
     Output('table-clients', 'columns')],
    [Input('segment-dropdown', 'value'),
     Input('store-rfm', 'data')]
)
def update_dashboard(selected_segment, data):
    if data is None or selected_segment is None: return no_update

    df = pd.DataFrame(data)
    subset = df[df['Segmento'] == selected_segment]

    # KPIs
    kpi_r = [html.Div("RECENCIA PROM."), html.Div(f"{subset['Recency'].mean():.0f} días")]
    kpi_f = [html.Div("FRECUENCIA PROM."), html.Div(f"{subset['Frequency'].mean():.1f}")]
    kpi_m = [html.Div("TICKET PROM."), html.Div(f"${subset['Monetary'].mean():.0f}")]

    # Colores
    colors = {'Campeones (VIP)': '#10b981', 'Leales': '#3b82f6', 'Necesitan Atención': '#f59e0b', 'En Riesgo': '#ef4444'}
    color_discrete_map = {k: v for k, v in colors.items() if k in df['Segmento'].unique()}

    # Scatter
    fig_scat = px.scatter(df, x='Recency', y='Monetary', color='Segmento', log_y=True,
                          hover_data=['customerid', 'Frequency'], title="Mapa de Clientes (Recencia vs Valor)",
                          color_discrete_map=color_discrete_map, template='plotly_dark')
    fig_scat.update_traces(marker=dict(opacity=0.3))
    # Highlight
    fig_scat.add_traces(px.scatter(subset, x='Recency', y='Monetary').update_traces(marker=dict(color='white', size=8, line=dict(width=2, color='black'))).data)
    fig_scat.update_layout(paper_bgcolor=COLORS['card'], plot_bgcolor=COLORS['card'], font_color=COLORS['text'])

    # Bar Chart
    counts = df['Segmento'].value_counts().reset_index()
    counts.columns = ['Segmento', 'Clientes']
    counts['Color'] = counts['Segmento'].apply(lambda x: colors.get(x, '#cccccc'))
    counts['Opacity'] = counts['Segmento'].apply(lambda x: 1.0 if x == selected_segment else 0.3)

    fig_bar = go.Figure(data=[go.Bar(
        x=counts['Segmento'], y=counts['Clientes'],
        marker_color=counts['Color'], marker_opacity=counts['Opacity']
    )])
    fig_bar.update_layout(title="Distribución de Segmentos", paper_bgcolor=COLORS['card'], plot_bgcolor=COLORS['card'], font_color=COLORS['text'])

    # Estrategia
    strategies = {
        'Campeones (VIP)': {'Acción': 'Acceso Anticipado', 'Canal': 'Email VIP', 'Mensaje': 'Sé el primero en ver lo nuevo.'},
        'Leales': {'Acción': 'Upselling', 'Canal': 'App Push', 'Mensaje': 'Descuento por volumen.'},
        'Necesitan Atención': {'Acción': 'Reactivación', 'Canal': 'Email Automatizado', 'Mensaje': 'Te extrañamos, aquí tienes $10.'},
        'En Riesgo': {'Acción': 'Recuperación Agresiva', 'Canal': 'SMS', 'Mensaje': '¡Vuelve! 50% OFF hoy.'}
    }
    strat = strategies.get(selected_segment, {'Acción': '-', 'Canal': '-', 'Mensaje': '-'})

    strat_html = html.Table([
        html.Thead(html.Tr([html.Th("Acción Clave"), html.Th("Canal"), html.Th("Mensaje")])),
        html.Tbody(html.Tr([html.Td(strat['Acción']), html.Td(strat['Canal']), html.Td(strat['Mensaje'])]))
    ], style={'width': '100%', 'color': '#cbd5e1'})

    # Tabla
    cols = [{"name": i, "id": i} for i in ['customerid', 'Recency', 'Frequency', 'Monetary']]

    return kpi_r, kpi_f, kpi_m, fig_scat, fig_bar, strat_html, subset.head(10).to_dict('records'), cols

# 6. EJECUCIÓN
# mode='inline' muestra la app dentro de la celda de salida
app.run(port=8050)
```

```
⚙️ Instalando librerías necesarias... (Esto puede tardar un minuto)
Dash is running on http://127.0.0.1:8050/

INFO:dash.dash:Dash is running on http://127.0.0.1:8050/

 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:8050
INFO:werkzeug:Press CTRL+C to quit
```